A Project Report on

"Nyāya Logic as an Inference Engine"

Submitted by:

Mr. Sanjeev Kushwaha

In the partial fulfilment for the award of the degree

of

Bachelor of Science

in

Computer Science

Under the guidance of

Asst. Prof. Sini. V.

SMT. DEVKIBA MOHANSINHJI CHAUHAN

COLLEGE OF COMMERCE & SCIENCE, SILVASSA

(Affiliated to University of Mumbai)

Academic Year: 2026–27



SMT. DEVKIBA MOHANSINHJI CHAUHAN

COLLEGE OF COMMERCE & SCIENCE, SILVASSA

(Affiliated to University of Mumbai)

SILVASSA – UT OF DD & DNH – 396230

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that **Mr. Sanjeev Kushwaha** of **T.Y.B.Sc. CS (Semester - V)** class has satisfactorily completed the Project **"Nyāya Logic as an Inference Engine",** to be submitted in the partial fulfilment for the award of **Bachelor of Science in Computer Science** during the academic year **2026–2027**.

Date of Submission:

Project Guide **Signature of Examiner**

College Seal **Coordinator**

DECLARATION

I, **Sanjeev Kushwaha**, hereby declare that the project entitled **"Nyāya Logic as an Inference Engine"** submitted in partial fulfilment for the award **of Bachelor of Science in Computer Science** during the academic **year 2026-2027** is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship, or any other similar titles.

Signature of the Student: _____

Place: _____

Date: _____

Table of Contents

1. Problem Statement 1
1.1 IKS Concept 1
1.2 CS Concept 2
1.3 Problem Statement 2
2. Conceptual Mapping and System Architecture 3
2.1 Conceptual Mapping Table 3
2.2 System Architecture 4
3. Algorithm Specification 5
3.1 Input 5
3.2 Output 5
3.3 Pseudo-code 5
3.4 Nyāya Mapping in Algorithm 6
4. Working Code 7-11
5. Test Cases and Detailed Test Results 12
5.1 Test Case Table 12
5.2 Detailed Test Results 13-15
6. Complexity Analysis and Limitations 16
6.1 Complexity Analysis 16
6.2 Space Complexity 16
6.3 Limitations 17
7. Conclusion 18

TOPIC: Nyāya Logic as an Inference Engine

1. Problem Statement

1.1 IKS Concept

Nyāya is one of the six orthodox (*āstika*) schools of Indian philosophy. Its foundational text, the *Nyāya Sūtra*, is traditionally attributed to Gautama (Akṣapāda). The school is mainly concerned with logic, epistemology, and the methods of reasoning. In Nyāya, valid knowledge (*pramā*) is obtained through a *pramāṇa*, a means of valid knowledge. The tradition recognises four *pramāṇas*:

1. **Pratyakṣa** – perception
2. **Anumāna** – inference
3. **Upamāna** – comparison
4. **Śabda** – verbal testimony

This project concentrates on **anumāna**, or inference. Nyāya presents a well-formed inference as a five-membered argument called **Panchāvayava Vākya**, whose members are:

1. **Pratijñā** (proposition): the statement to be established.
2. **Hetu** (reason): the ground offered in support of the proposition.
3. **Udāharaṇa** (example): a general rule illustrated by a familiar instance.
4. **Upanaya** (application): the application of the general rule to the case under discussion.
5. **Nigamana** (conclusion): the statement of the established conclusion.

The standard textbook illustration is the inference of fire on a hill from the presence of smoke:

Member	Statement
Pratijñā	The hill has fire.
Hetu	Because the hill has smoke.
Udāharaṇa	Wherever there is smoke, there is fire, as in a kitchen.
Upanaya	This hill has smoke of that kind—smoke pervaded by fire.
Nigamana	Therefore, the hill has fire.

1.2 CS Concept

A **rule-based expert system** stores knowledge as facts and IF-THEN production rules. Its **inference engine** repeatedly matches rule conditions against the facts held in **working memory** and adds the consequents of matching rules as new facts. When the process starts from known facts and moves towards conclusions, it is called **forward chaining**. The engine stops when a complete pass produces no new fact.

The two ideas share a broad pattern: from what is already known (facts), a general relation (a rule) is applied to reach a new statement (a conclusion). This report uses that pattern as a computational analogy. It does **not** claim that Nyāya reasoning is identical to forward chaining. Nyāya inference is a philosophical account of valid knowledge that also deals with doubt, fallacious reasons (*hetvābhāsa*), and debate, none of which is modelled here.

1.3 Problem Statement

To design and implement a small **Nyāya-inspired rule-based inference engine** that:

- accepts a set of initial facts from the user;
- holds a predefined knowledge base of IF-THEN production rules;
- applies the rules by forward chaining to derive new facts;
- reports all derived facts and the **terminal derived facts** (the end points of the inference chains); and
- illustrates, through a simplified mapping, how selected Nyāya concepts correspond to components of a rule-based system.

The implementation is an **educational demonstration** of the relationship between an Indian Knowledge Systems (IKS) topic and rule-based Artificial Intelligence. It is **not** intended for commercial or diagnostic use.

2. Conceptual Mapping and System Architecture

2.1 Conceptual Mapping Table

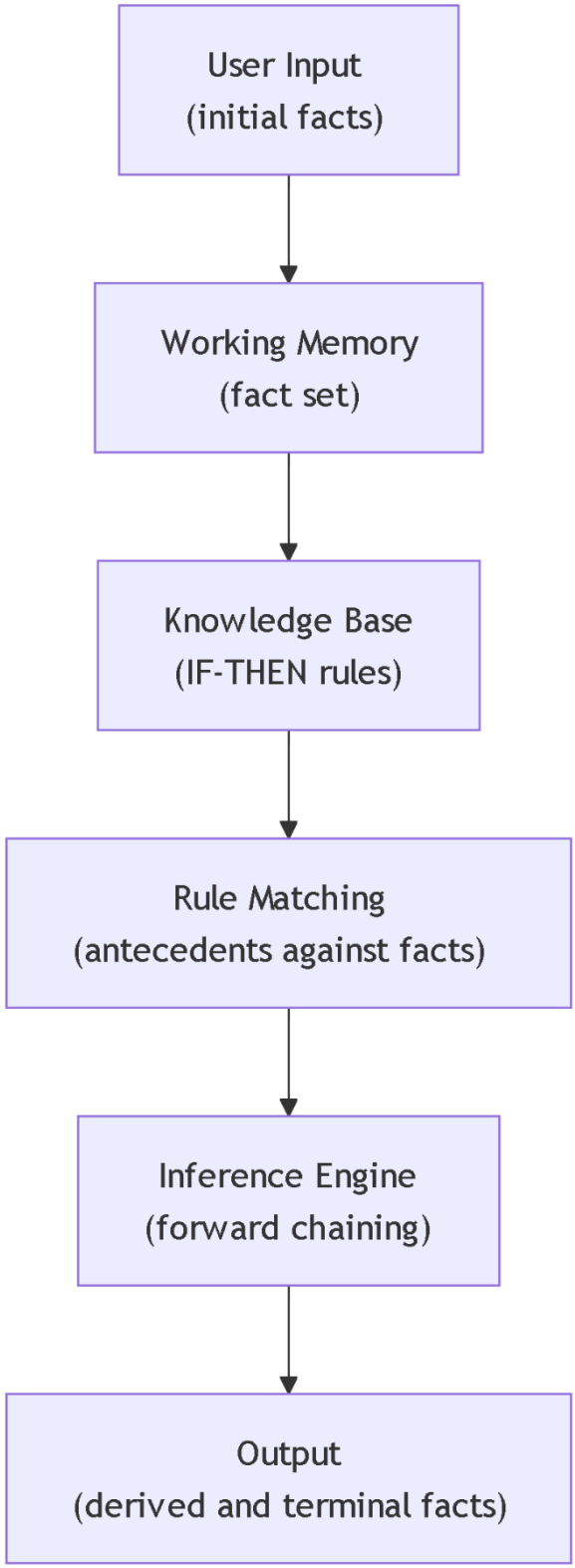
Table 1 lists the correspondences used in this project. Each mapping is **approximate**: the Nyāya terms have philosophical meanings that a software component can represent only in a limited way. The remarks column states the main point of difference.

Table 1: Mapping between Nyāya concepts and CS representations:

Nyāya Concept	CS Representation Used	Remarks
Pramāṇa (means of valid knowledge)	Facts and rules the engine is allowed to rely on (knowledge base and input)	Pramāṇa is a means of knowing, not a data store. The knowledge base is only a rough counterpart.
Pratyakṣa (perception)	Initial facts entered by the user	Only perception-like input is modelled. Facts are assumed to be true.
Anumāna (inference)	Inference engine (forward chaining)	The engine models only the step of deriving new statements from known ones.
Vyāpti (invariable concomitance)	IF–THEN production rule	Nyāya requires an invariable relation. Here, a rule is an assumed implication, and some rules are simplified.
Pakṣa (locus of inference)	Working memory (current set of facts)	Loose analogy. Pakṣa is the specific subject, such as the hill.
Hetu (reason)	Rule antecedent (IF part)	Nyāya also examines whether a hetu is valid. The engine does not.
Sādhyā (what is to be established)	Rule consequent (THEN part)	—
Pañcāvayava Vākya	Sequence of goal, facts, rule, matching, and derived fact	Used for comparison only. See Appendix A.
Nigamana (conclusion)	Derived fact / program output	—
Tarka (hypothetical reasoning)	Backward chaining	Not implemented. Mentioned for comparison.
Nirṇaya (settled conclusion)	No direct equivalent	Termination at a fixed point is a property of the algorithm and is not the meaning of nirṇaya.

2.2 System Architecture

Figure 1 shows the flow of data through the system. New facts produced by the inference engine are written back to working memory, so that later rules can use them.



3. Algorithm Specification

3.1 Input

A set of initial facts, for example smoke, clouds, or wet_ground, entered as comma-separated names.

3.2 Output

- The complete set of known facts.
- The list of derived facts in the order of derivation.
- The number of iterations.
- The **terminal derived facts**. A terminal derived fact is a derived fact that was **not used as a condition** by any rule that fired during the run.

3.3 Pseudo-code:

```
ALGORITHM Nyaya_Inference_Engine
START

1.  Initialize Working Memory (fact set) = { }
2.  Initialize Knowledge Base (rule set) = { }
3.  Initialize Derived Facts = [ ]
4.  Initialize Used Facts = { }
5.  Initialize iteration = 0

6.  Read initial facts from the user
7.  Add all initial facts to Working Memory

8.  REPEAT
9.      iteration = iteration + 1
10.     Set new_fact_derived = FALSE
11.     FOR each rule in Knowledge Base DO
12.         IF all antecedents of the rule are in Working Memory THEN
13.             IF the consequent is NOT in Working Memory THEN
14.                 Add consequent to Working Memory
15.                 Append consequent to Derived Facts
16.                 Add all antecedents of the rule to Used Facts
17.                 Set new_fact_derived = TRUE
18.             END IF
19.         END IF
20.     END FOR
21. UNTIL new_fact_derived = FALSE

22. Compute Terminal Derived Facts = Derived Facts - Used Facts
23. Display all facts in Working Memory
24. Display all derived facts in order
25. Display iteration count
26. Display the terminal derived facts

END
```

3.4 Nyāya Mapping in Algorithm

Table 2: Algorithm steps and corresponding Nyāya concepts:

Step	Algorithm Action	Nyāya Concept (Analogy)
4–5	Read the initial facts into working memory	**Pratyakṣa** (perception-like input)
6–7	Start a pass over the rules	**Pakṣa** (current subject of inference), loosely
8–9	Test whether the antecedents of a rule hold	**Vyāpti** (the rule) and **Hetu** (the reason)
10–13	Add the consequent as a new fact	**Upanaya** (application of the rule) leading to a conclusion
17	Stop when a pass derives nothing new	Termination condition of the algorithm. No Nyāya equivalent is claimed.
18–20	Display derived and terminal facts	**Nigamana** (conclusion)

4. Working Code

The program is written in **Python 3** and uses no external libraries. It is saved as nyaya_engine.py and run with python **nyaya_engine.py**. Rules are checked in the order listed, and a fact added during a pass is available to the rules that follow it in the same pass. The final output reports **terminal derived facts**, which are the derived facts that no fired rule used as a condition.

nyaya_engine.py-

```
# ============================================================
# Nyaya Logic as an Inference Engine
# Forward-Chaining Rule-Based Expert System
# ============================================================

# ------------------------------------------------------------
# KNOWLEDGE BASE (Vyapti - Universal Rules)
# Each rule is of the form: (antecedents_list, consequent)
# Antecedents are stored as a list; a single-antecedent rule
# still uses a list for uniformity.
# ------------------------------------------------------------

RULES = [
    (["smoke"], "fire"),
    (["fire"], "heat"),
    (["heat"], "high_temperature"),
    (["clouds"], "rain"),
    (["rain"], "wet_ground"),
    (["rain"], "cold_wind"),
    (["cold_wind"], "cold_weather"),
    (["cold_weather"], "wear_warm_clothes"),
    (["smoke", "hill_has_smoke"], "hill_has_fire"),
    (["fire"], "light"),
    (["light", "night"], "visibility_possible"),
    (["wet_ground", "sun_shining"], "water_evaporating"),
    (["water_evaporating"], "high_humidity"),
    (["high_humidity", "dark_clouds"], "rain_likely"),
    (["rain_likely"], "carry_umbrella"),
    (["high_temperature", "high_humidity"], "humid_weather"),
]

# ------------------------------------------------------------
# FORWARD-CHAINING INFERENCE ENGINE
# ------------------------------------------------------------

def forward_chain(initial_facts, rules, verbose=True):
    """
    Forward-chaining inference engine.
    Derives new facts from initial facts using IF-THEN rules.

    Returns:
        working_memory : set of all known facts
        derived_facts  : list of derived facts in order
        conclusions    : list of terminal derived facts
        iterations     : number of passes made
    """
```

```
    working_memory = set(initial_facts)
    derived_facts = []
    used_facts = set()
    iteration = 0

    if verbose:
        print("\n" + "=" * 60)
        print("NYAYA INFERENCE ENGINE - FORWARD CHAINING")
        print("=" * 60)
        print("\n[Initial Facts - Pratyaksha]")
        for fact in sorted(working_memory):
            print(f"  * {fact}")

    while True:
        iteration += 1
        new_fact_derived = False
        newly_added = []

        if verbose:
            print(f"\n--- Iteration {iteration} ---")

        for number, (antecedents, consequent) in enumerate(rules, 1):
            if all(ant in working_memory for ant in antecedents):
                if consequent not in working_memory:
                    working_memory.add(consequent)
                    derived_facts.append(consequent)
                    used_facts.update(antecedents)
                    newly_added.append((number, antecedents, consequent))
                    new_fact_derived = True

        if verbose:
            if newly_added:
                print("[New Facts Derived - Upanaya & Nigamana]")
                for number, ant, cons in newly_added:
                    print(f"  Rule {number}: IF {ant} THEN {cons}")
            else:
                print("  No new facts derived.")

        if not new_fact_derived:
            break


    # Terminal derived facts: derived facts not used as any antecedent
    conclusions = [f for f in derived_facts if f not in used_facts]

    return working_memory, derived_facts, conclusions, iteration
```

```
# ------------------------------------------------------------
# OUTPUT / DISPLAY MODULE
# ------------------------------------------------------------

def display_results(working_memory, derived_facts, conclusions, iterations):
    print("\n" + "=" * 60)
    print("FINAL RESULTS - NIGAMANA (CONCLUSION)")
    print("=" * 60)

    print("\n[All Known Facts]")
    for fact in sorted(working_memory):
        print(f"  * {fact}")

    print(f"\n[Total Facts]: {len(working_memory)}")
    print(f"[Derived Facts]: {len(derived_facts)}")
    print(f"[Iterations]: {iterations}")

    if derived_facts:
        print("\n[Newly Derived Facts]")
        for fact in derived_facts:
            print(f"  + {fact}")

    print("\n[Final Conclusion(s)]")
    if conclusions:
        for c in conclusions:
            print(f"  [OK] {c}")
    else:
        print("  No final conclusion could be derived.")


# ------------------------------------------------------------
# MAIN PROGRAM
# ------------------------------------------------------------

def main():
    print("NYAYA LOGIC AS AN INFERENCE ENGINE")
    print("IKS Concept as CS Concept")
    print("=" * 60)

    print("\nAvailable facts you can use:")
    all_facts = set()
    for ant, cons in RULES:
        all_facts.update(ant)
        all_facts.add(cons)
    for fact in sorted(all_facts):
        print(f"  - {fact}")

    print("\nEnter initial facts separated by commas.")
    print("Example: smoke, hill_has_smoke, clouds")
    print("Type 'exit' to quit.\n")

    user_input = input("Enter facts: ").strip().lower()
    if user_input == "exit":
        print("Exiting...")
        return
```

```
    initial_facts = [f.strip() for f in user_input.split(",") if f.strip()]
    if not initial_facts:
        print("No facts entered. Exiting.")
        return

    working_memory, derived_facts, conclusions, iterations = forward_chain(
        initial_facts, RULES
    )
    display_results(working_memory, derived_facts, conclusions, iterations)


# ------------------------------------------------------------
# ENTRY POINT
# ------------------------------------------------------------

if __name__ == "__main__":
    main()
```

OUTPUT:

```
   NYAYA LOGIC AS AN INFERENCE ENGINE
   IKS Concept as CS Concept
============================================================

Available facts you can use:
  - carry_umbrella
  - clouds
  - cold_weather
  - cold_wind
  - dark_clouds
  - fire
  - heat
  - high_humidity
  - high_temperature
  - hill_has_fire
  - hill_has_smoke
  - humid_weather
  - light
  - night
  - rain
  - rain_likely
  - smoke
  - sun_shining
  - visibility_possible
  - water_evaporating
  - wear_warm_clothes
  - wet_ground

Enter initial facts separated by commas.
Example: smoke, hill_has_smoke, clouds
(Type 'exit' to quit)

Enter facts: smoke,hill_has_smoke
```

```
============================================================
NYAYA INFERENCE ENGINE - FORWARD CHAINING
============================================================

[Initial Facts - Pratyaksha]
  * hill_has_smoke
  * smoke

--- Iteration 1 ---
[New Facts Derived - Upanaya & Nigamana]
  Rule 1: IF ['smoke'] THEN fire
  Rule 2: IF ['fire'] THEN heat
  Rule 3: IF ['heat'] THEN high_temperature
  Rule 8: IF ['smoke', 'hill_has_smoke'] THEN hill_has_fire
  Rule 9: IF ['fire'] THEN light

--- Iteration 2 ---
  No new facts derived.

============================================================
FINAL RESULTS - NIGAMANA (CONCLUSION)
============================================================

[All Known Facts]
  * fire
  * heat
  * high_temperature
  * hill_has_fire
  * hill_has_smoke
  * light
  * smoke

[Total Facts]: 7
[Derived Facts]: 5
[Iterations]: 2

[Newly Derived Facts]
  -> fire
  -> heat
  -> high_temperature
  -> hill_has_fire
  -> light

[Final Conclusion(s)]
  [OK] high_temperature
  [OK] hill_has_fire
  [OK] light
```

5. Test Cases and Results

5.1 Test Case Table

Twelve test cases were executed to verify the program.

- Initial Facts: Facts given by the user.
- Derived Facts: Facts generated by applying rules.
- Terminal Facts: Final derived facts shown as conclusions.

Since new facts are added immediately, rule chains are completed in the first iteration. The second iteration checks for new facts and terminates when none are found.

Table 3: Test cases and results obtained from the implementation

Test	Initial Facts	Derived Facts (in order)	Terminal Derived Facts
TC01	smoke	fire, heat, high_temperature, light	high_temperature, light
TC02	clouds	rain, wet_ground	wet_ground
TC03	smoke, hill_has_smoke	fire, heat, high_temperature, hill_has_fire, light	high_temperature, hill_has_fire, light
TC04	rain, cold_wind	wet_ground, cold_weather, wear_warm_clothes	wet_ground, wear_warm_clothes
TC05	wet_ground, sun_shining	water_evaporating, high_humidity	high_humidity
TC06	high_humidity, dark_clouds	rain_likely, carry_umbrella	carry_umbrella
TC07	light, night	visibility_possible	visibility_possible
TC08	fire, high_humidity	heat, high_temperature, light, humid_weather	light, humid_weather
TC09	clouds, cold_wind	rain, wet_ground, cold_weather, wear_warm_clothes	wet_ground, wear_warm_clothes
TC10	smoke, clouds, sun_shining	fire, heat, high_temperature, rain, wet_ground, light, water_evaporating, high_humidity, humid_weather	light, humid_weather
TC11	rain, dark_clouds, sun_shining	wet_ground, water_evaporating, high_humidity, rain_likely, carry_umbrella	carry_umbrella
TC12	night, cold_wind	None	None

5.2 Detailed Test Results

TC01 — Single chain from one fact

Initial facts: smoke

Iteration 1:

- Rule 1 → fire
- Rule 2 → heat
- Rule 3 → high_temperature
- Rule 10 → light

Iteration 2: No new fact; the engine stops.

Terminal derived facts: high_temperature, light

TC02 — Short chain (clouds to wet ground)

Initial facts: clouds

Iteration 1:

- Rule 4 → rain
- Rule 5 → wet_ground

Iteration 2: No new fact; the engine stops.

Terminal derived facts: wet_ground

TC03 — Rule with two antecedents (Rule 9) together with a chain

Initial facts: smoke, hill_has_smoke

Iteration 1:

- Rule 1 → fire
- Rule 2 → heat
- Rule 3 → high_temperature
- Rule 9 → hill_has_fire
- Rule 10 → light

Iteration 2: No new fact; the engine stops.

Terminal derived facts: high_temperature, hill_has_fire, light

TC04 — Chain through rain and cold wind

Initial facts: rain, cold_wind

Iteration 1:

- Rule 5 → wet_ground
- Rule 7 → cold_weather
- Rule 8 → wear_warm_clothes

Iteration 2: No new fact; the engine stops.

Terminal derived facts: wet_ground, wear_warm_clothes

TC05 — Chain through evaporation to humidity

Initial facts: wet_ground, sun_shining

Iteration 1:

- Rule 12 → water_evaporating
- Rule 13 → high_humidity

Iteration 2: No new fact; the engine stops.

Terminal derived facts: high_humidity

TC06 — Two-antecedent rule (Rule 14) followed by a chain

Initial facts: high_humidity, dark_clouds

Iteration 1:

- Rule 14 → rain_likely
- Rule 15 → carry_umbrella

Iteration 2: No new fact; the engine stops.

Terminal derived facts: carry_umbrella

TC07 — Two-antecedent rule (Rule 11) only

Initial facts: light, night

Iteration 1:

- Rule 11 → visibility_possible

Iteration 2: No new fact; the engine stops.

Terminal derived facts: visibility_possible

TC08 — Chain from fire and high humidity

Initial facts: fire, high_humidity

Iteration 1:

- Rule 2 → heat
- Rule 3 → high_temperature
- Rule 10 → light
- Rule 16 → humid_weather

Iteration 2: No new fact; the engine stops.

Terminal derived facts: light, humid_weather

TC09 — Two independent inputs giving a chain

Initial facts: clouds, cold_wind

Iteration 1:

- Rule 4 → rain
- Rule 5 → wet_ground
- Rule 7 → cold_weather
- Rule 8 → wear_warm_clothes

Iteration 2: No new fact; the engine stops.

Terminal derived facts: wet_ground, wear_warm_clothes

TC10 — Independent chains that merge at Rule 16

Initial facts: smoke, clouds, sun_shining

Iteration 1:

- Rule 1 → fire
- Rule 2 → heat
- Rule 3 → high_temperature
- Rule 4 → rain
- Rule 5 → wet_ground
- Rule 10 → light
- Rule 12 → water_evaporating
- Rule 13 → high_humidity
- Rule 16 → humid_weather

Iteration 2: No new fact; the engine stops.

Terminal derived facts: light, humid_weather

TC11 — Chain through rain, dark clouds, and sun

Initial facts: rain, dark_clouds, sun_shining

Iteration 1:

- Rule 5 → wet_ground
- Rule 12 → water_evaporating
- Rule 13 → high_humidity
- Rule 14 → rain_likely
- Rule 15 → carry_umbrella

Iteration 2: No new fact; the engine stops.

Terminal derived facts: carry_umbrella

TC12 — Negative test: no rule applies

Initial facts: night, cold_wind

Iteration 1: No rule matches, so no new fact is derived.

Terminal derived facts: None

6. Complexity Analysis and Limitations

6.1 Time Complexity

Let:

- *r* = number of rules
- *a* = average number of antecedents per rule
- *f* = number of facts in working memory
- *i* = number of iterations until no new fact is derived

In every iteration, each rule is examined once. Checking a rule tests each of its antecedents against working memory, which is a hash set with average O(1) membership tests. One iteration therefore costs O(r × a), and the total time is:

O(i × r × a)

Every iteration except the last adds at least one new fact, so i ≤ d + 1, where *d* is the number of derived facts and $d \leq f$. In the worst case the time is therefore O(f × r × a).

For the supplied program, *r* = 16, *a* ≤ 2, and the vocabulary has a fixed number of distinct facts. The running time can be called constant, O(1), only under the assumption that the rule and fact universe is a fixed, small demonstration set. If rules or facts are added, the running time grows as given above.

6.2 Space Complexity

Working memory and the list of derived facts each require **O(f)** space, where *f* is the number of facts in working memory, since every known fact is stored once in the fact set and every derived fact is stored once in the ordered list. The rule base requires **O(r × a)** space, where *r* is the number of rules and *a* is the average number of antecedents per rule, since every rule stores its antecedents as a list. The total space required by the program is therefore **O(f + r × a)**, which is linear in the number of facts and rules and remains small for the fixed demonstration set supplied in this project.

6.3 Limitations

1. Simplified representation of Nyāya. Nyāya includes sixteen categories (*padārthas*), four *pramāṇas*, and a theory of debate. Only a simplified view of *anumāna* is represented here.
2. Hand-written rules. The rules are fixed in the program. Some are simplified; for example, clouds → rain is not universally true, so it would not qualify as a *vyāpti* in the Nyāya sense. Rule 9 (smoke + hill_has_smoke → hill_has_fire) is also redundant, since Rule 1 already derives fire from smoke.
3. No validity checking of reasons. The engine does not test whether a *hetu* is faulty (*hetvābhāsa*). Every matching rule is accepted.
4. No uncertainty. Facts are either present or absent. Doubt (*saṃśaya*) and probabilistic reasoning are not modelled.
5. No backward chaining. Only data-driven forward chaining is implemented. Goal-driven reasoning, which is closer to the way a *pratijñā* is stated first, is not.
6. Limited explanation. The program shows the rule that produced each fact, but it does not produce a full five-step Nyāya argument for each conclusion.
7. Fixed rule order and no conflict handling. Rules fire in the order listed. The rule base contains no contradictory conclusions, so no conflict-resolution strategy is needed or implemented.
8. No learning, time, or multiple agents. Facts are static, rules are not learned, and debate (*vāda*) between several reasoners is not represented.
9. Educational purpose only. The program has not been validated for any real-world use.

7. Conclusion

This project set out to design and implement a small Nyāya-inspired rule-based inference engine and to use it as a concrete demonstration of how selected concepts from an Indian Knowledge System (IKS) can be mapped onto components of a classical Artificial Intelligence technique. The implementation accepts initial facts from the user, holds a predefined knowledge base of IF–THEN production rules, and applies forward chaining to derive new facts until no further fact can be produced. The conceptual mapping presented in Section 2.1 shows that initial facts correspond loosely to *pratyakṣa*, rules to *vyāpti*, rule antecedents to *hetu*, and derived facts to *nigamana*, while also stating clearly where the analogy is approximate and does not hold.

The program was tested against twelve test cases covering single-fact chains, multi-antecedent rules, independent chains that merge, and a negative case where no rule applies. In every case, the derived facts matched the expected results, and the terminal derived facts were computed correctly. The time and space complexity analysis in Section 6 shows that the engine runs in **O(i × r × a)** time and **O(f + r × a)** space, and for the fixed demonstration set supplied in this project, these values are small and the program runs quickly. The main limitations are the small hand-written rule base, the absence of any check on the validity of a reason (*hetvābhāsa*), and the absence of backward chaining and uncertainty handling.

In conclusion, the project shows that Nyāya logic and rule-based inference engines, though developed in very different contexts and for very different purposes, share a recognizable pattern: from what is already known, a general relation is applied to reach a new statement. The project does not claim that the two are identical, nor that Nyāya can be reduced to a computer program. Rather, it uses the analogy as a pedagogical bridge, helping students of computer science appreciate the logical sophistication of Indian Knowledge Systems, and helping students of IKS see how classical ideas can inspire and inform modern computational thinking. The work is offered as an educational demonstration, not as a diagnostic or commercial tool, and it is hoped that it will encourage further exploration at the intersection of Indian philosophy and Artificial Intelligence.